REPORT

Reviewer report: urllib3/urllib3#5367: Allow omitted

filenames in multipart file tuple type annotations

Run urllib3-5367, tested 2026-11-09 11:52 IST by ops/daily_run.py (route source-pair).
Proof-of-Change commit

ef12096677e9cf59e8673436d74ec65d53771d7c. Report generated 2026-11-09 18:38 IST by

reports/pr_report.py.

Baseline: full (test comparison + claim probes + coverage gate + claim flagging (differential probes shown,

informational)).

NEEDS ATTENTION

Recommended action: Reject Confidence: High

The run is trustworthy and found a difference a person should look at. Every check passed and every claim cites

a log line.

Advice only: Proof-of-Change never blocks a merge and never posts on GitHub.

Summary in plain English

• What it says it does: Allow omitted filenames in multipart file tuple type annotations (urllib3/urllib3#5367).

• What we observed: probe: 4 observation(s) need attention: c1.annotation does not differ although the claim

says it does (C1); c2.annotation does not differ although the claim says it does (C2); c3.annotation does not differ

although the claim says it does (C3); c5.annotation does not differ although the claim says it does (C5); probe:

claim(s) C4 proven; tests: 20 test(s) ran on main, 24 on the PR; 4 new test(s) pass, 20 unchanged.

• What you should do: Do not approve as is. Read 'What changed in behavior' and decide with the PR author.

Demo helper

These notes are background from the console and the daily check. They do not change the verdict.

If setup fails, the helper retries the install up to 2 times. If a test still can't be verified because something is

missing or fails to import, it looks up the cause, installs the fix on both sides and reruns that step, up to 3 tries.

• Setup check: GitHub token: present.

• Setup check: Nebius token: present.

• Setup check: free memory 6.4 GB (of 15.6 GB).

• Setup check: free disk 92.8 GB.

• Demo helper: git is installed.

• Demo helper: python3 is installed.

• Demo helper: uv is installed.

• Looked up with Tavily: [redacted] results

• Query: urllib3 python setup install instructions https://github.com/urllib3/urllib3 site:github.com/urllib3/urllib3

• Query: urllib3 python documentation install https://github.com/urllib3/urllib3 site:github.com/urllib3/urllib3

• Query: urllib3 python install requirements https://pypi.org/project/urllib3/ https://github.com/urllib3/urllib3

site:pypi.org

• Source: urllib3 · PyPI — https://pypi.org/project/urllib3/

• Source: urllib3 installation requires setuptools >= 36.2.5 · Issue #1753 · urllib3/urllib3 · GitHub —

https://github.com/urllib3/urllib3/issues/1753

• Source: urllib3/docs/user-guide.rst at main · urllib3/urllib3 · GitHub —

https://github.com/urllib3/urllib3/blob/main/docs/user-guide.rst

• Source: ImportError during install of urllib3[secure] on Jython · Issue #1133 · urllib3/urllib3 · GitHub —

https://github.com/urllib3/urllib3/issues/1133

• Source: GitHub - urllib3/urllib3: urllib3 is a user-friendly HTTP client library for Python · GitHub —

https://github.com/urllib3/urllib3

• Source: Installing urllib3 · Issue #829 · urllib3/urllib3 · GitHub — https://github.com/urllib3/urllib3/issues/829

• Source: No module named 'urllib3.packages.six' · Issue #1518 · urllib3/urllib3 · GitHub —

https://github.com/urllib3/urllib3/issues/1518

• Setup plan: the same packages on main and pr.

• Setup plan: install trustme>=1.2.1 (listed in the test extra) on main and pr.

• Setup plan: Python 3.12: Python 3.12 already meets >=3.10

• Installed trustme 1.2.1 from pyproject (listed in the test extra) on main and pr.

Setup: pytest plugin asyncio was turned off. Pytest stopped at startup the same way on both sides, and the

traceback pointed into that third-party plugin (AttributeError: module 'pytest' has no attribute

'pyodide_options_stack'). The tests were run again with the plugin turned off. This does not change how the

verdict is chosen. Detection fired. Pytest stopped at startup the same way on both sides. Plugin chosen: asyncio.

The installed pytest plugin list was read. Retry flags: -p no:asyncio. Outcome: worked.

Claims vs reality

| # | Claim (source) | Status | What we observed | Log line

| C2 | Existing behavior is kept: tests that pass on main still pass (implied by every PR) | proven | no test that

passes on main fails on the PR (20 unchanged, 3 runs per side) | pytest-pr-1.log:77 «24 passed in 2.61s»

| P-C1 | The change allows omitting filenames (using None) in multipart file tuple type annotations. (PR

description) | not observed | probe model-p1, c1.annotation: same on both sides:

"_TYPE_FIELD_VALUE_TUPLE" | probe-model-p1-pr-1.jsonl:1 «"observation": "c1.annotation"»

| P-C2 | Calls like (None, b"{}", "application/json") fail type checking in RequestField.from_tuples() and

encode_multipart_formdata() on unmodified main. (PR description) | not observed | probe model-p1,

c2.annotation: same on both sides: "_TYPE_FIELD_VALUE_TUPLE" | probe-model-p1-pr-1.jsonl:2

«"observation": "c2.annotation"»

| P-C3 | After the patch, both tuple forms accept str | None as the filename. (PR description) | not observed |

probe model-p1, c3.annotation: same on both sides: "_TYPE_FIELD_VALUE_TUPLE" |

probe-model-p1-pr-1.jsonl:3 «"observation": "c3.annotation"»

| P-C5 | The public API example had two mypy errors on unmodified main and no errors with the patch, while

executing successfully both before and after. (PR description) | not observed | probe model-p1, c5.annotation:

same on both sides: "_TYPE_FIELD_VALUE_TUPLE" | probe-model-p1-pr-1.jsonl:4 «"observation":

"c5.annotation"»

| P-C4 | Parameterized tests verify that for two- and three-item forms with string and bytes data, the emitted

Content-Disposition and Content-Type headers are correct. (PR description) | proven | probe model-p2,

c4.case1.content_disposition: same on both sides: "form-data; name=\"testfield\"" | probe-model-p2-pr-1.jsonl:1

«"observation": "c4.case1.content_disposition"»

Citation check (PASS): 6 citation(s), every one re-read from its log file and matched.

What changed in behavior

| What | Count | Examples

| Tests that actually ran (main / PR, fewest of the runs) | 20 / 24 | | Tests unchanged | 20 | | New tests that pass | 4 | | Tests that pass on main but fail on the PR | 0 | | Tests that are new in the PR and fail | 0 | | Tests that ran on main but are missing on the PR side | 0 | | Tests that changed result in another way (for example passed -> skipped) | 0 | | Tests that gave different results between the

3 runs of the same side (flaky) | 0 | | Tests that fail on main and pass on the PR | 0 | | Tests that are new or changed but did not run because an optional module is missing | 0 | | Tests that were re-parametrized (same test, new parameter ids; not counted as removed) | 0 | | Test modules that fail to load on the PR side only (their tests no longer run) | 0 | | Test modules that fail to load on main only (no baseline for their tests) | 0 | | Test modules that fail to load on both sides (their tests did not run) | 0 | | Tests that fail identically on both sides (already broken on main) | 0 | -

Probes

Model-written probes: model provider: nebius (Token Factory, live).

A probe is a small deterministic script run on both sides in the same environments as the tests, 3 alternating runs

per side. It prints JSON observations; plain code (poc/diff.py) compares them. A difference that no claim of the

PR covers needs attention. Every probe was frozen (content sha256) before any PR-side code ran, and checked

against that freeze before each PR-side run. Only a live model probe is blind; fixed probes and recorded model

fixtures are known-answer cases.

model-p1 (model (nvidia/nemotron-3-super-120b-a12b, live), model-probes/model-p1.py, frozen 102cca783acc

at 2026-11-09T06:25:37.259607+00:00): Probe for C1,C2,C3,C5: Check annotation of

RequestField.from_tuples.value

• P-C1 (PR description): The change allows omitting filenames (using None) in multipart file tuple type

annotations. Expects the covered observations to differ.

• P-C2 (PR description): Calls like (None, b"{}", "application/json") fail type checking in

RequestField.from_tuples() and encode_multipart_formdata() on unmodified main. Expects the covered

observations to differ.

• P-C3 (PR description): After the patch, both tuple forms accept str | None as the filename. Expects the covered

observations to differ.

• P-C5 (PR description): The public API example had two mypy errors on unmodified main and no errors with the

patch, while executing successfully both before and after. Expects the covered observations to differ.

| Observation | Claim | Result | What the probe saw (main -> PR) | Log lines (main, PR)

| c1.annotation | P-C1 | no change although the claim says it changes | same on both sides:

"_TYPE_FIELD_VALUE_TUPLE" | probe-model-p1-main-1.jsonl:1 «"observation":

"c1.annotation"»
probe-model-p1-pr-1.jsonl:1 «"observation": "c1.annotation"»

| c2.annotation | P-C2 | no change although the claim says it changes | same on both sides:

"_TYPE_FIELD_VALUE_TUPLE" | probe-model-p1-main-1.jsonl:2 «"observation":

"c2.annotation"»
probe-model-p1-pr-1.jsonl:2 «"observation": "c2.annotation"»

| c3.annotation | P-C3 | no change although the claim says it changes | same on both sides:

"_TYPE_FIELD_VALUE_TUPLE" | probe-model-p1-main-1.jsonl:3 «"observation":

"c3.annotation"»
probe-model-p1-pr-1.jsonl:3 «"observation": "c3.annotation"»

| c5.annotation | P-C5 | no change although the claim says it changes | same on both sides:

"_TYPE_FIELD_VALUE_TUPLE" | probe-model-p1-main-1.jsonl:4 «"observation":

"c5.annotation"»
probe-model-p1-pr-1.jsonl:4 «"observation": "c5.annotation"»

model-p2 (model (nvidia/nemotron-3-super-120b-a12b, live), model-probes/model-p2.py, frozen e1883660191a

at 2026-11-09T06:25:37.259767+00:00): Probe for C4: Check headers for two- and three-item forms with string

and bytes data and filename=None

• P-C4 (PR description): Parameterized tests verify that for two- and three-item forms with string and bytes data,

the emitted Content-Disposition and Content-Type headers are correct. Expects the covered observations to stay

the same.

| Observation | Claim | Result | What the probe saw (main -> PR) | Log lines (main, PR)

| c4.case1.content_disposition | P-C4 | as claimed | same on both sides: "form-data; name=\"testfield\"" |

probe-model-p2-main-1.jsonl:1 «"observation": "c4.case1.content_disposition"»
probe-model-p2-pr-1.jsonl:1

«"observation": "c4.case1.content_disposition"»

| c4.case1.content_type | P-C4 | as claimed | same on both sides: "application/octet-stream" |

probe-model-p2-main-1.jsonl:2 «"observation": "c4.case1.content_type"»
probe-model-p2-pr-1.jsonl:2

«"observation": "c4.case1.content_type"»

| c4.case2.content_disposition | P-C4 | as claimed | same on both sides: "form-data; name=\"testfield\"" |

probe-model-p2-main-1.jsonl:3 «"observation": "c4.case2.content_disposition"»
probe-model-p2-pr-1.jsonl:3

«"observation": "c4.case2.content_disposition"»

| c4.case2.content_type | P-C4 | as claimed | same on both sides: "application/octet-stream" |

probe-model-p2-main-1.jsonl:4 «"observation": "c4.case2.content_type"»
probe-model-p2-pr-1.jsonl:4

«"observation": "c4.case2.content_type"»

| c4.case3.content_disposition | P-C4 | as claimed | same on both sides: "form-data; name=\"testfield\"" |

probe-model-p2-main-1.jsonl:5 «"observation": "c4.case3.content_disposition"»
probe-model-p2-pr-1.jsonl:5

«"observation": "c4.case3.content_disposition"»

| c4.case3.content_type | P-C4 | as claimed | same on both sides: "text/plain" | probe-model-p2-main-1.jsonl:6

«"observation": "c4.case3.content_type"»
probe-model-p2-pr-1.jsonl:6 «"observation":

"c4.case3.content_type"»

| c4.case4.content_disposition | P-C4 | as claimed | same on both sides: "form-data; name=\"testfield\"" |

probe-model-p2-main-1.jsonl:7 «"observation": "c4.case4.content_disposition"»
probe-model-p2-pr-1.jsonl:7

«"observation": "c4.case4.content_disposition"»

| c4.case4.content_type | P-C4 | as claimed | same on both sides: "application/json" |

probe-model-p2-main-1.jsonl:8 «"observation": "c4.case4.content_type"»
probe-model-p2-pr-1.jsonl:8

«"observation": "c4.case4.content_type"»

Citation check (PASS): 24 citation(s), every one re-read from its log file and matched.

Was the changed code exercised?

Coverage: MEASURED by coverage.py on the PR side (7.0 s run).

Coverage gate (python, confidence normal): 1 of 1 changed executable line(s) ran (100%).

| Changed file | Changed executable lines | Of those, ran

| src/urllib3/fields.py | 1 | 1

How this is measured: one extra, untimed run of the PR side's tests under coverage.py. Changed lines come

from comparing the tested main and PR files. Compiled code (C, C++, Cython) is invisible to coverage.py; such a

change only counts as verified when the PR adds or changes a test for that module.

What went right

• Tested 88d3fbea7cc3 of the PR against main (source-pair route).

• 3 alternating runs per side: main 3, PR 3 of 3.

• pytest exit codes explained: all runs exited 0.

• Changed code was exercised (coverage gate): 1 of 1 changed executable line(s) ran (100%).

• Probes ran on both sides: 2 probe(s), 3 alternating runs per side, every run printed its observations.

• Single-thread environment: OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1,

MKL_NUM_THREADS=1, NUMEXPR_NUM_THREADS=1, VECLIB_MAXIMUM_THREADS=1,

BLIS_NUM_THREADS=1, PYTHONHASHSEED=0.

• Machine type recorded: local x86_64 8 cpus.

• Nothing posted upstream: GitHub was only read.

• Known-answer replay (fresh re-run): replayed 2026-10-10 06:30:21 IST, harness 1.0.

• Every claim cites a real log line: 6 citation(s), every one re-read from its log file and matched.

• Every probe observation cites its output line: 24 citation(s), every one re-read from its log file and matched.

What went wrong and how to fix it

| Issue | Status | Detail | How to fix

| Behavior difference found | finding | probe: 4 observation(s) need attention: c1.annotation does not differ

although the claim says it does (C1); c2.annotation does not differ although the claim says it does (C2);

c3.annotation does not differ although the claim says it does (C3); c5.annotation does not differ although the

claim says it does (C5); probe: claim(s) C4 proven; tests: 20 test(s) ran on main, 24 on the PR; 4 new test(s)

pass, 20 unchanged | This is about the PR, not the run: read the failing tests and their log lines and the Probes

section, then decide.

Evals and guardrails

| Check | Result | Detail

| 3 alternating runs per side | PASS | main 3, PR 3 of 3

| pytest exit codes explained | PASS | all runs exited 0

| Changed code was exercised (coverage gate) | PASS | 1 of 1 changed executable line(s) ran (100%)

| Probes ran on both sides | PASS | 2 probe(s), 3 alternating runs per side, every run printed its observations

| Single-thread environment | PASS | OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1,

MKL_NUM_THREADS=1, NUMEXPR_NUM_THREADS=1, VECLIB_MAXIMUM_THREADS=1,

BLIS_NUM_THREADS=1, PYTHONHASHSEED=0

| Machine type recorded | PASS | local x86_64 8 cpus

| Nothing posted upstream | PASS | GitHub was only read

| Known-answer replay (fresh re-run) | PASS | replayed 2026-10-10 06:30:21 IST, harness 1.0

| Every claim cites a real log line | PASS | 6 citation(s), every one re-read from its log file and matched

| Every probe observation cites its output line | PASS | 24 citation(s), every one re-read from its log file and

matched

Checklist

• I read the verdict and the plain-English summary.
• Every 'false' or 'unmentioned change' row has a log line I can open.
• Nothing under 'What went wrong' is a run problem (those mean Re-run, not Reject).
• I recorded my decision (see below).

Decision recorded: none yet. To record one, run make decide REPORT=poc-main/reports/daily/2026-11-09/urllib3-5367 DECISION=approve (or reject, approve_with_notes, re_run).

Rerun command

make check PR=https://github.com/urllib3/urllib3/pull/5367

Evidence links

• Pull request urllib3/urllib3#5367
• record.json
• pr.diff
• setup-main.log
• setup-pr.log
• git.log
• pytest-main-1.log
• pytest-pr-1.log
• pytest-main-2.log
• pytest-pr-2.log
• pytest-main-3.log
• pytest-pr-3.log
• junit-main-1.xml
• junit-pr-1.xml
• coverage-pytest.log
• coverage-pr.json
• probe-model-p1-main-1.jsonl
• probe-model-p1-pr-1.jsonl
• probe-model-p2-main-1.jsonl
• probe-model-p2-pr-1.jsonl
• main run 1 result: pytest-main-1.log:77 «20 passed in 2.66s»
• pr run 1 result: pytest-pr-1.log:77 «24 passed in 2.61s»

Glossary

| Term | Meaning

| main | The library as it is today, without the PR.

| PR | Pull request: a proposed change to the library.

| run | One full pass of the selected tests. We do 3 per side, alternating main and PR.

| flaky | A test that gives different results when nothing changed; it is ignored as evidence.

| log line | A line in a pytest-*.log file: the raw output the verdict is based on.

| route | How the two sides were built: nightly+patch, source-build or source-pair (see docs/OPERATIONS.md).

| changed lines covered | How many of the PR's changed, executable Python lines ran during the PR side's tests.

| re-parametrized | A test whose parameter ids changed (renamed cases); the new ids are judged on their own.

rebuilt from cleaned text

Written 2026-11-09 18:38 IST